

[Lab1-v1.4]-Configuracao de Interoperabilidade entre sistemas e troca de mensagens por servicos RestFul. -Final Combate.

data: 08/12.09.2025

status: *in progress*

Pre-requisito ambiente local:

- e. Lab0-1.2.1-Jsr317 implementado e funcional

S2A3. Crie as seguintes configuracoes de seguranca baseada na arquitectura Centralizada de recursos.

a. EndpointDispatcherConfiguration - e a entidade que permite centralizar todos os recursos/servicos disponiveis e rotea-os para serem consumidos apartir de um ponto central **"/api"**. Sua implemental pode ser vista abaixo.

```
14  /**
15  *
16  * @author shifu
17  */
18  @ApplicationPath(EndpointDispatcherConfiguration.ROOT_PATH)
19  public class EndpointDispatcherConfiguration extends javax.ws.rs.core.Application {
20
21      public static final String ROOT_PATH = "/api";
22      public static final String HOSTNAME = "http://localhost:8080";
23
24      private Set<Class<?>> endpoints = new HashSet<>();
25
26      /**
27       * add endpoints dynamically. Just once at load.
28       *
29       * @return
30       */
31      @Override
32      public Set<Class<?>> getClasses() {
33
34          this.endpoints.add(AuthenticationEndpoint.class);
35          this.endpoints.add(UserAuthenticationEndpoint.class);
36
37          return this.endpoints;
38      }
39
40  }
```

b.

c. **E'** notavel os endpoints relacionadas **AuthenticationEndpoint** que controla todo o fluxo de autenticacao e autorizacao de um pedido

```
33  /** @author shifu
34  */
35  @Path(AuthenticationEndpoint.RESOURCE_NAME)
36  public class AuthenticationEndpoint implements AuthenticationType {
37
38      public static final String RESOURCE_NAME = "/oauth";
39
40      private final UserDao userDao;
41      private java.util.logging.Logger logger = java.util.logging.Logger.getLogger(this.getClass().getName());
42
43
44      public AuthenticationEndpoint() throws ClassNotFoundException, SQLException {
45
46          this.userDao = new UserDaoImpl();
47
48          //adding resource info
49          logger.log(Level.SEVERE, "moHitekframework v1.0.0 - Binding URL=[{0}] into Path=[{1}].",
50              new Object[]{HOSTNAME, RESOURCE_NAME});
51      }
52  }
```

d.

e. atraves do servico **basicAuthentication**, o client frontEnd, sera capaz de identificar se o pedido de autenticacao ocorreu com sucesso.
Request.ok()

```

103 | @Override
104 | public Response basicAuth(@PathParam("username") final String username, @PathParam("password") final String pas
105 |
106 |     UserEntity user = null;
107 |     //buscar o objecto apartir do username
108 |     try {
109 |
110 |         user = userDao.readUserEntity(username);
111 |
112 |         //authentication phase
113 |         if(user !=null && password.equals(user.getCredential())){
114 |
115 |             //authorization phase
116 |             //https://datatracker.ietf.org/doc/html/rfc6749
117 |
118 |             //set information Store.
119 |             final NewCookie newCk = new NewCookie(new Cookie("session_id", UUID.randomUUID().toString()),null,
120 |                 10,
121 |                 null,
122 |                 false,false);
123 |
124 |             return Response.ok("Access Granted to user ".concat(user.getName()))
125 |                 .header("Authentication", "Basic")
126 |                 .header("Content-Type" , MediaType.TEXT_PLAIN)
127 |                 .cookie(newCk)
128 |                 .build();
129 |         }
130 |
131 |     } catch (ClassNotFoundException | SQLException | NullPointerException e) {

```

f.

```

djo@opensource:~$ curl -i http://127.0.0.1:8080/trainee25xxdemo210/api/auth/basicAuth/trainee24xxshifu/da
HTTP/1.1 200 OK
Server: Apache-Coyote/1.1
Authorization: Bearer dHJhaw5lZTI0eHhzaGldTpkYQ==
Set-Cookie: session_id=37c739fd-624d-4603-bbbf-06cff79afda7;Version=1;Max-Age=10
Content-Type: text/plain
Content-Length: 39
Date: Tue, 02 Sep 2025 08:07:28 GMT

Access Granted to user Trainee24xxShifu
djo@opensource:~$

```

g.

caso nao uma mensagem de erro e' retornada como resposta

```

    //authentication failure.
    return Response.status(Status.FORBIDDEN)
        .build();
}

```

h.

i. Garantidos os passos de **autenticacao**, o pedido passa para o **estagio de authorization** atraves do endpoint

```

17 | @Path(UserAuthenticationEndpoint.RESOURCE_NAME)
18 | public class UserAuthenticationEndpoint {
19 |
20 |     public final static String RESOURCE_NAME = "/user";
21 |
22 |     private final UserDao userDao;
23 |     private java.util.logging.Logger logger = java.util.logging.Logger.getLogger(this.getClass().getName());
24 |
25 |     /*
26 |     HTTP Status 500 - A MultiException has 1 exceptions.
27 |     */
28 |     public UserAuthenticationEndpoint() throws ClassNotFoundException, SQLException {
29 |
30 |         this.userDao = new UserDaoImpl();
31 |     }
32 |
33 |     @Path("/findUser/{username}")
34 |     @GET
35 |     public Response readUser(@PathParam("username") final String username ) {
36 |
37 |         //identificar o usuario na base de dados
38 |         UserEntity userEntity = null;
39 |
40 |
41 |         try {
42 |
43 |             userEntity = this.userDao.readUserEntity(username);
44 |
45 |             //1.definir o formato
46 |             String token64 = username.concat("**").concat(userEntity.getCredential());

```

j.

k. Notem que o usuario precisa de ter acesso a todos os perfis acoplados a ele. Desta forma precisamos adicionar uma lista na entidade UserEntity.



- l.
e seus respectivos metodos de encapsulamento. Apenas o **getProfiles()**
m.